

۱. طراحی و پیاده سازی واحد حساب و منطق^۱ ۳۲ بیتی

هدف از این تمرین، آشنایی عملی با نحوه عملکرد واحد حساب و منطق، به عنوان قلب تپنده پردازنده است. در این تمرین، شما یک واحد حساب و منطق ترکیبی ۳۲ بیتی طراحی خواهید کرد که قادر به انجام عملیات جمع، تفریق و ضرب است. در طراحی این واحد حساب و منطق، عملیات جمع باید با استفاده از پیمانه^۲ جمع کننده با انتشار رقم نقلی^۳ پیاده سازی شود. عملیات تفریق نیز باید با جمع متمم دو^۴ عدد دوم با عدد اول به دست آید. همچنین، در این تمرین مجاز به استفاده از روش های ترتیبی نیستید و عملیات ها باید به صورت کاملاً ترکیبی پیاده سازی شوند.

مشخصات ورودی ها و خروجی ها

پیمانه شما باید دارای خطوط ورود/خروج زیر باشد:
ورودی ها:

- A: عملوند اول (۳۲ بیت)
- B: عملوند دوم (۳۲ بیت)
- ALU_Ctrl: سیگنال کنترلی برای تعیین نوع عملیات (۳ بیت)

خروجی ها:

- Result_Lo: ۳۲ بیت کم ارزش حاصل عملیات
- Result_Hi: ۳۲ بیت پر ارزش حاصل عملیات، فقط در عملیات ضرب کاربرد دارد و در سایر دستورات صفر است.
- C (Carry): پرچم رقم نقلی
- V (Overflow): پرچم سرریز
- Z (Zero): پرچم صفر، اگر خروجی صفر باشد، این بیت ۱ می شود.
- N (Negative): پرچم منفی، نشان دهنده علامت نتیجه در محاسبات علامت دار است.

^۱ALU

^۲Module

^۳Carry Ripple Adder

^۴2's Complement

منطق پیاده‌سازی

سیگنال کنترل واحد حساب و منطق^۵ تعیین‌کننده عملیاتی است که واحد حساب و منطق باید انجام دهد. بیت‌های این سیگنال را به این صورت در نظر بگیرید:

- بیت پرارزش^۶ (تعیین نوع عملیات): ۰ برای جمع/تفریق و ۱ برای ضرب.
- بیت میانی (تعیین جمع یا تفریق): ۰ برای جمع و ۱ برای تفریق.
- بیت کم‌ارزش^۷ (تعیین علامت‌دار بودن): ۰ برای بدون علامت و ۱ برای علامت‌دار.

جدول ۱ کدهای کنترلی واحد حساب و منطق را نشان می‌دهد.

جدول ۱: کدهای کنترلی واحد حساب و منطق

کد کنترلی ^۸	نام دستور	شرح عملیات	پرچم‌های فعال
000	ADD_U	جمع دو عدد بدون علامت ($A + B$)	C, Z, V
001	ADD_S	جمع دو عدد علامت‌دار ($A + B$)	V, N, Z
010	SUB_U	تفریق دو عدد بدون علامت ($A - B$)	C, Z, V
011	SUB_S	تفریق دو عدد علامت‌دار ($A - B$)	V, N, Z
100	MUL_U	ضرب ترکیبی دو عدد بدون علامت ($A \times B$)	Z
سایر حالت‌ها	NOP	خروجی‌ها صفر باشند.	هیچ‌کدام

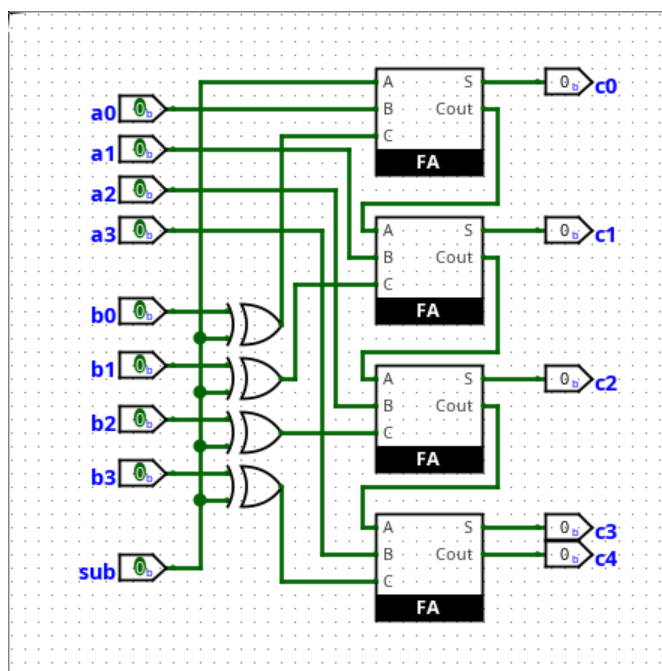
حین پیاده‌سازی این مدارها نیز، برای سهولت، از مدارهای دارای ورودی رشته‌ای استفاده کنید. زیرا استفاده از مدارها و بلوک‌های رشته‌ای از تکرار مدار جلوگیری می‌کند. منظور از بلوک رشته‌ای، المان‌هایی مانند عملگرهای منطقی است که به‌جای ورودی یا خروجی تک‌بیتی، با رشته‌ای از بیت‌ها کار می‌کنند. برای رشته‌ای کردن یک گیت، تنها کافی است پارامتر Data Bits مربوط به آن را تغییر دهید.

برای درک بهتر الگوریتم و نحوه‌ی پیاده‌سازی، به شکل‌های ۱ و ۲ توجه کنید.

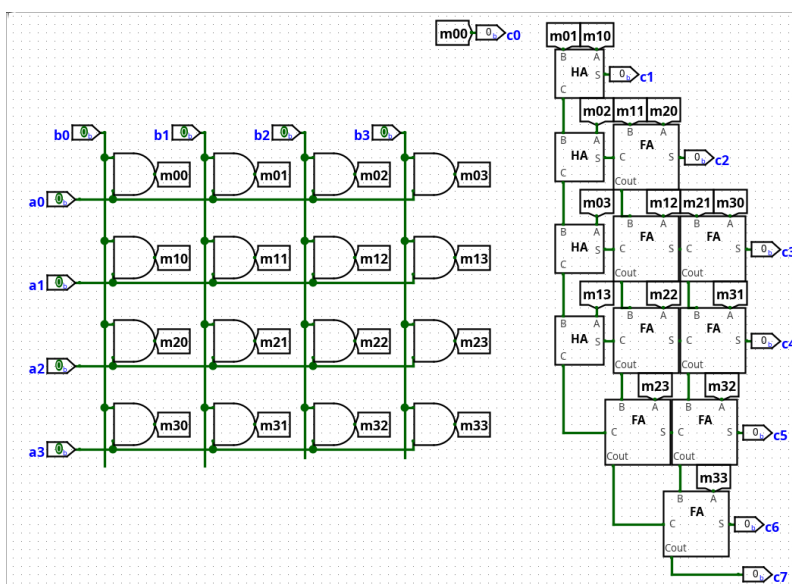
^۵ALU_Ctrl

^۶MSB

^۷LSB



شکل ۱: نمونه پیاده‌سازی جمع/تفریق‌کننده ۴ بیتی



شکل ۲: نمونه پیاده‌سازی ضرب‌کننده ترکیبی ۴ بیتی

خروجی‌های مورد انتظار برای تحویل

- فایل طراحی مدار در محیط Logisim (نام فایل: HW1-STUDENT_ID.circ).
- گزارش کاملی از نحوه طراحی و اجرای آزمون ارائه کنید (نام گزارش: HW1-STUDENT_ID.pdf). توجه داشته باشید که نمره‌دهی تنها بر اساس نتایج همین آزمون انجام می‌شود.